


# **L13a**: RESTful Web Services and APIs

HTTP requests, responses, and the CRUD methods

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

# Objectives for Today

How does our code get data from a program that runs on another computer?

## Today's concepts

- Describe web services and APIs: a web service lets machines interact over a network, and an API is the set of rules used to access the service.
- Read an HTTP request and response: a request has a method, a URL, headers, and a body. A response has a status code, headers, and a body.
- Explain how a RESTful API works: follow a request through a RESTful web service, and map the create, read, update, and delete operations onto the HTTP methods.

---

**Lecture:** RESTful Web Services and APIs

# Lecture and Example

**Lecture:** RESTful Web Services and APIs
In this lecture, we introduce web services and the application programming interfaces (APIs) that our programs use to call them. We focus on RESTful APIs, the most common design for web services, and on the HTTP methods that create, read, update, and delete data.

**Example:** Download a weather forecast from the National Weather Service API
In this example, we write a small Julia library, called a software development kit (SDK), for the RESTful API of the National Weather Service. We make HTTP GET calls to two endpoints and return the hourly forecast for Ithaca, NY as a table.

# Web Services and APIs

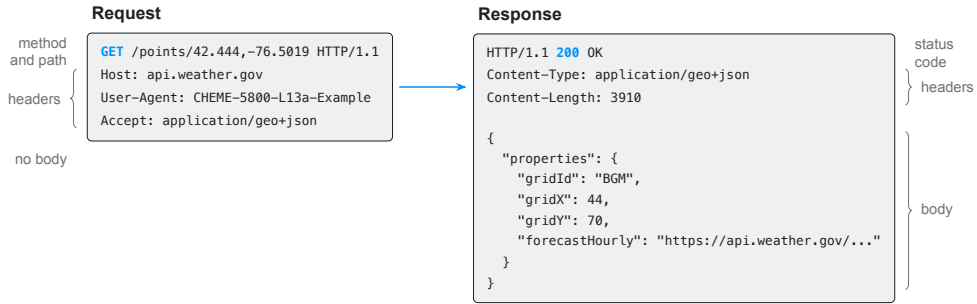
A web service is software that lets machines interact over a network. An application programming interface (API) is the set of rules used to access the service. The client makes a request, and the server answers it.

- API specification: the requests that a client can make and the data that comes back.
- Public and private APIs: clients call the public API methods, which in turn call the private API methods.
- Communication protocol: a shared set of rules for exchanging messages. Most web services use the Hypertext Transfer Protocol (HTTP), with data in formats such as JSON, CSV, and XML.

Most HTTP messages travel over TCP, which delivers the bytes reliably and in order. HTTPS is HTTP sent over a connection encrypted with Transport Layer Security (TLS).

# A Request and Its Response

HTTP is a request-response protocol. The client sends a request message, and the server answers with a response message.




A client asks the National Weather Service (NWS) API for the data that it holds about Ithaca, NY. The status code 200 says that the request succeeded.

# The HTTP Request

A request has four parts:

- Method: a verb that says what the client wants to do with the data, e.g., `GET` to retrieve it.
- Headers: name and value pairs that describe the request, e.g., the `User-Agent` header identifies the client, and the `Accept` header gives the data format that the client wants.
- Body: the data that the client sends to the server. The body is optional, and the `GET` requests in this lecture do not send one.
- URL: the Uniform Resource Locator, the address of the data on the server.

# The Pieces of a URL

A URL has four main pieces. The URL of the hourly weather forecast for Ithaca, NY from the NWS API, in metric units:




The URLs that an API publishes are called endpoints. The NWS API has one endpoint for the data about a location, and another endpoint for an hourly forecast.

# The HTTP Response

A response has three parts: a status code, the headers, and the body. The first digit of the status code gives the class of the response:

| Class | Meaning | Common codes |
|---|---|---|
| `2xx` | the request succeeded | `200 OK`, `201 Created`, `204 No Content` |
| `3xx` | the client must take another step | `301 Moved Permanently` |
| `4xx` | the client made an error | `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `429 Too Many Requests` |
| `5xx` | the server failed | `500 Internal Server Error`, `503 Service Unavailable` |

A `4xx` code means that the problem is with the request that the client sent. A `5xx` code means that the server could not process it.

# JSON

JavaScript Object Notation (JSON) is a text format for structured data. Most web services use JSON for the body of a request and the body of a response.

- Object: a set of `"key": value` pairs inside braces `{ }`.
- Array: an ordered list of values inside brackets `[ ]`.
- Value: a string, a number, `true`, `false`, `null`, or another object or array.

Parsing converts JSON text into data that a program can use. In our Julia code, a parsed object becomes a dictionary and a parsed array becomes a vector. Serialization converts the data in a program into text such as JSON.

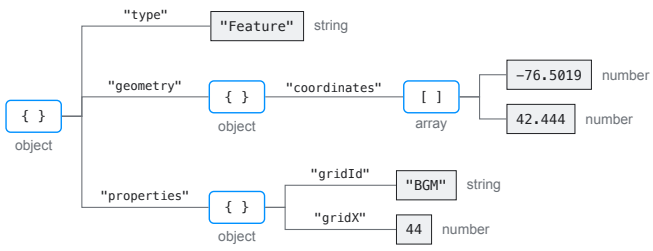
# JSON Text Is a Tree of Data

Part of the body of the NWS response, and the data that the text describes:

**JSON text**

```
{
  "type": "Feature",
  "geometry": {
    "coordinates": [-76.5019, 42.444]
  },
  "properties": {
    "gridId": "BGM",
    "gridX": 44
  }
}
```

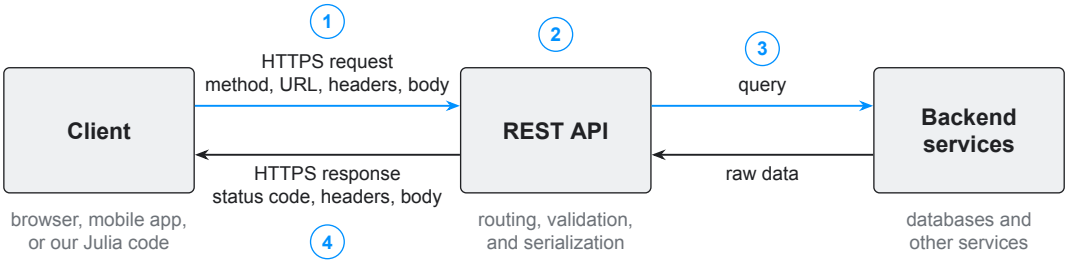
**Tree of data**




Objects and arrays can hold other objects and arrays. We reach the grid identifier `"BGM"` by following the `"properties"` key and then the `"gridId"` key.

# RESTful Web Services

Representational State Transfer (REST) is an architecture for creating stateless web services: each request carries all the information that the server needs. REST organizes a web service around resources, items of data such as a user, an experiment, or a weather forecast. Each resource has its own URL.



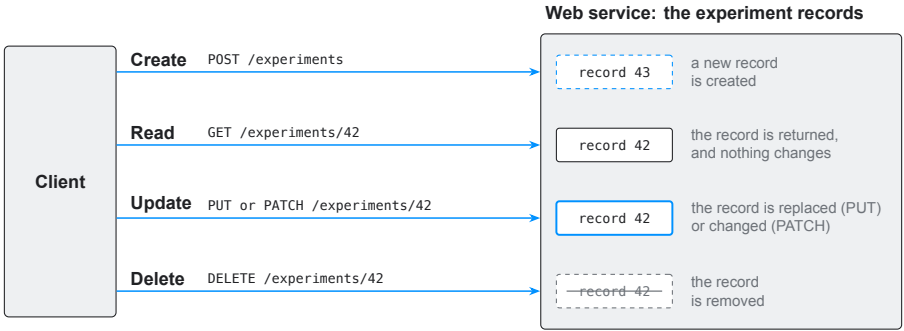

A typical RESTful web service answers a request in four steps. The client needs to know what to send and what to expect in return.

# The Four Steps of the Cycle

1. Client → REST API: the client sends an HTTPS request to an endpoint. It selects the HTTP method, and encodes any data in the body, typically as JSON.
2. REST API internals: the API maps the endpoint to a handler function (routing). The handler checks the client and its inputs (validation), parses the JSON in the request, and converts the result back into JSON (serialization).
3. REST API → backend services: the handler gets data from a database or another web service, often with a protocol other than HTTP.
4. REST API → client: the API converts the result into a JSON response, attaches a status code, and sends the response back over HTTPS.

# HTTP Methods and CRUD

Most of what a client does with a resource is one of four operations: create, read, update, or delete (CRUD). A RESTful API maps each operation onto an HTTP method.




A lab runs a web service for its experiment records. A `GET` request leaves the records unchanged, and the other requests change them.

# What the Server Does for Each Method

The lab's web service for its experiment records, with the status code that the server returns on success:

| Operation | Example request | What the server does | Success code |
|---|---|---|---|
| Create | `POST /experiments` | creates a new record from the JSON in the body | `201 Created` |
| Read | `GET /experiments/42` | returns record `42`, and changes nothing | `200 OK` |
| Update | `PUT /experiments/42` | replaces record `42` with the JSON in the body | `200 OK` |
| Update | `PATCH /experiments/42` | changes only the fields that the body lists | `200 OK` |
| Delete | `DELETE /experiments/42` | removes record `42`, and returns no body | `204 No Content` |

The URL `/experiments` is the collection of all the records. The URL `/experiments/42` is the record with identifier `42`.

# Safe and Idempotent Methods

- Safe: the method only reads data. Of the five methods, only `GET` is safe.
- Idempotent: sending the same request many times has the same effect on the server as sending it once. `GET`, `PUT`, and `DELETE` are idempotent.
- `POST` is not idempotent: in our example, sending the same `POST` request twice creates two records.

A client can send an idempotent request again after a timeout.

In Julia, the HTTP.jl package has one function for each method:

```
HTTP.post(url, headers, body)   # create
HTTP.get(url, headers)          # read
HTTP.put(url, headers, body)    # update
HTTP.patch(url, headers, body)  # update
HTTP.delete(url, headers)       # delete
```

Each returns a response with `status`, `headers`, and `body` fields.

# Why Are RESTful Web Services so Popular?

RESTful web services have become the standard for building web APIs. Three principles of the architecture explain why:

- Statelessness: each request is complete on its own, and the server does not need to remember earlier requests from the client. Any one of many servers can answer the next request.
- Uniform interface: a small, well-known set of methods operates on resource URLs. A developer who has used one RESTful API can quickly understand another.
- Layered system: the client does not need to know whether it is talking to a single program or to a chain of services. The developers can change the backend without changing the client.

# Example: The National Weather Service API

In this example, we write a small Julia library, called a software development kit (SDK), and download the hourly forecast for Ithaca, NY:

1. Build the URL of the points endpoint from a latitude and longitude, and make an HTTP GET call.
2. Read the URL of the hourly forecast from the response, and call the forecast endpoint.
3. Pass a handler to the forecast call that returns the forecast as a table.

The NWS API publishes resources that a client can read but not change. Every call in the example uses the `GET` method.

---

**Example:** A software development kit for the National Weather Service API

# Summary

We introduced web services, the HTTP messages that carry their calls, and the REST architecture.

## Key takeaways

- **Web services and APIs.** A client needs to know what to send to an endpoint and what to expect in return.
- **HTTP requests and responses.** A request has a method, a URL, headers, and a body. The status code of the response says whether the request succeeded.
- **REST and the CRUD methods.** Each resource has its own URL, and POST, GET, PUT or PATCH, and DELETE create, read, update, and delete it.

These ideas are the foundation for calling any web service from our own code.